



BURSA TEKNİK ÜNİVERSİTESİ

BURSA TEKNİK ÜNİVERSİTESİ

Bilgisayar Mühendisliği Bölümü

BLM0326 - Bilgisayar Ağları Dönem Projesi

Proje Başlığı:

**NetProbe: UDP Tabanlı Güvenilir Dosya Aktarımı ve Ağ Performans
Analiz Platformu**

GRUP 18

Grup Üyeleri:

23360859726 – Selenay Bulut

23360859057 – Elif Kara

22360859070 – Rumeysa Ersoy

Hazırlanma Tarihi: Haziran 2026

İÇİNDEKİLER

1. Giriş

- 1.1. Projenin Amacı ve Taşıma Katmanı Kavramları
- 1.2. Proje Görev Dağılımı

2. Problem Tanımı

- 2.1. Paket Kaybı (Packet Loss)
- 2.2. Sıra Bozulması (Out-of-Order Delivery)
- 2.3. Veri Bozulması (Data Corruption) ve İkili/JSON Uyumsuzluğu
- 2.4. Çift Yazma (Duplicate Write) Problemi

3. Sistem Mimarisi

- 3.1. protocol/packet.py – Protokol Katmanı
- 3.2. client/client.py – İstemci Katmanı
- 3.3. server/server.py – Sunucu Katmanı
- 3.4. analysis/analyzer.py ve main.py Orkestratörü

4. Protokol Tasarımı

- 4.1. Stop-and-Wait ARQ Protokolü
- 4.2. JSON Tabanlı Header Yapısı
- 4.3. Base64 Kodlamasının Gerekliliği

5. Gerçekleme Detayları

- 5.1. Python Socket API Kullanımı
- 5.2. CHUNK_SIZE = 1024 Bayt Seçiminin Gerekçesi
- 5.3. TIMEOUT_VAL = 1.0 Saniye Kalibrasyonu
- 5.4. MAX_RETRIES = 5 Parametresi

6. Deney Ortamı

- 6.1. LOSS_RATE = %20 Simülasyonu
- 6.2. Log Analizi Yaklaşımı

7. Performans Metrikleri

- 7.1. Throughput (Verim)
- 7.2. Goodput (Etkin Verim)
- 7.3. Protokol Verimliliği ve Overhead

8. Sonuçlar ve Tartışma

- 8.1. Ölçüm Sonuçları
- 8.2. Overhead Analizi ve Trade-off Değerlendirmesi

9. Karşılaşılan Sorunlar ve Çözüm Yaklaşımları

- 9.1. İkili Veri / JSON Uyumsuzluğu Sorunu
- 9.2. Kayıp ACK Nedeniyle Çift Yazma Problemi
- 9.3. Zaman Aşımı Ayarının Kalibrasyonu

10. Sonuç ve Gelecekte Yapılabilecek Geliştirmeler

- 10.1. Sliding Window Protokolüne Geçiş
- 10.2. Çoklu İş Parçacığı (Multi-threading) Mimarisi
- 10.3. Uyarlanabilir Zaman Aşımı (Adaptive RTO)
- 10.4. Şifreleme ve Güvenlik Katmanı

11. Kaynakça

1.Giriş

1.1 Projenin Amacı ve Taşıma Katmanı Kavramları

Bilgisayar ağları, günümüz dijital altyapısının temel taşı oluşturmakta; veri iletişiminin güvenilir, hızlı ve verimli biçimde gerçekleştirilmesi, modern yazılım sistemlerinin tasarımında belirleyici bir unsur olarak öne çıkmaktadır. Bu bağlamda, OSI (Open Systems Interconnection) referans modelinin taşıma katmanı, uçtan uca veri iletiminin kalitesini ve bütünlüğünü doğrudan etkileyen kritik protokolleri barındırmaktadır. OSI modelinin 4. katmanı olan taşıma katmanı; veri parçalama (segmentation), akış kontrolü (flow control), hata denetimi ve çoğullama (multiplexing) işlevlerini üstlenmekte; bu katmanda tanımlanan iki temel protokol olan TCP (Transmission Control Protocol) ile UDP (User Datagram Protocol) ise birbirinden köklü biçimde farklılaşan tasarım felsefeleri sergilemektedir.

TCP, RFC 793 standardında tanımlanmış olup bağlantı yönelimli (connection-oriented) yapısıyla güvenilir, sıralı ve hata denetimli veri iletimini garanti altına almaktadır. Üçlü el sıkışma (threeway handshake) mekanizması sayesinde her oturum açılmakta; kayan pencere (sliding window) algoritması ile akış ve tıkanıklık kontrolü sağlanmakta; alıcı tarafından gönderilen onay (ACK) paketleri aracılığıyla kayıp veya bozulan segmentler yeniden iletilmektedir. Buna karşın, UDP RFC 768 ile tanımlanmış olup bağlantısız (connectionless) ve en iyi çabayı esas alan (best-effort) bir iletişim modeli benimser. UDP başlığı yalnızca kaynak port, hedef port, uzunluk ve sağlama toplamı (checksum) alanlarından oluşan 8 baytlık minimal yapısıyla, TCP'nin bağlantı kurulum gecikmelerini ve protokol ek yükünü tamamen ortadan kaldırmaktadır.

Bu bağlamda, gerçekleştirilen "NetProbe: UDP Tabanlı Güvenilir Dosya Aktarımı ve Ağ Performans Analiz Platformu" projesi; UDP'nin doğasında var olan güvensizlik problemini uygulama katmanında çözmeyi hedefleyen akademik bir sistem tasarımı çalışması olarak konumlandırılmaktadır. Proje kapsamında, Stop-and-Wait ARQ (Automatic Repeat reQuest) protokolü Python programlama diliyle sıfırdan gerçekleştirilmiş; dizi numarası (sequence number) ve onay numarası (ACK) mekanizmaları entegre edilmiş; ikili veri iletimini JSON paket yapısıyla uyumlu kılmak amacıyla Base64 kodlaması kullanılmıştır. Sistem, ağ trafiğini CSV formatında loglayan bir izleme altyapısına sahip olmakla birlikte, throughput ve goodput metriklerinin hesaplanmasına ve karşılaştırmalı görselleştirmesine olanak tanıyan bir performans analiz modülünü de bünyesinde barındırmaktadır.

Projenin akademik özgünlüğünü pekiştiren temel unsur, yalnızca işlevsel bir dosya aktarım uygulaması geliştirilmekle yetinilmeyip; ağ mühendisliği kavramlarının (paket kaybı simülasyonu, dayanıklılık testi, overhead analizi) sistematik biçimde ölçülmesi ve raporlanmasıdır. Bu yaklaşım, mevcut akademik literatürdeki QUIC (RFC 9000), SCTP (RFC 4960) ve RUDP gibi UDP üzeri güvenilir aktarım protokolleriyle örtüşen bir araştırma eksenini işaret etmektedir.

1.2 Proje Görev Dağılımı

Rumeysa Ersoy (Çekirdek Ağ ve Protokol Altyapısı):

Sistemin veri aktarım motorunu inşa etmiştir. UDP soket iletişimini kurmuş ve güvensiz UDP altyapısını güvenilir hale getiren Stop-and-Wait protokolünü kodlamıştır. Paketlerin sırasını (Sequence Number), karşıya ulaştığını (ACK) ve yolda bozulmadığını (Checksum) denetleyen tüm veri bütünlüğü mekanizmalarını (protocol/packet.py) sisteme entegre etmiştir.

Elif Kara (Ağ İzleme, Loglama ve Simülasyon):

Sistemin hareketlerini kayıt altına alan ve dayanıklılığını test eden mekanizmaları kurmuştur. Ağ trafiğini anlık olarak kaydeden asenkron loglama (CSV) sistemini geliştirmiş ve sunucu tarafına sistemi zorlayıp dayanıklılığını ölçecek yapay paket kaybı (LOSS_RATE) simülasyonunu (server/server.py) eklemiştir.

Selenay Bulut (Performans Analizi ve Veri Görselleştirme):

Projenin başarısını matematiksel verilerle ispatlayan analiz modülünü geliştirmiştir. Elde edilen log verilerini işleyerek sistemin brüt hızı olan Throughput ile net veri hızı olan Goodput metriklerini hesaplamış ve bu sonuçları Matplotlib kullanarak profesyonel çubuk grafiklere (analysis/analyzer.py) dönüştürerek raporlamıştır.

2. Problem Tanımı

UDP'nin çekici performans özellikleri, beraberinde ciddi güvenilirlik sorunlarını da getirmektedir. RFC 768'in açıkça ifade ettiği üzere, UDP herhangi bir bağlantı kurulumu, sıra garantisi veya teslim onayı sağlamaksızın veri birimlerini (datagram) ağa bırakmaktadır. Bu temel tasarım kararı, üç ayrı kategoride somutlaşan problematik durumlar ortaya çıkarmaktadır.

2.1 Paket Kaybı (Packet Loss)

IP ağlarında yönlendiriciler ve anahtarlar, tampon bellek (buffer) taşması, tıkanıklık veya bağlantı arızası durumlarında UDP datagramlarını sessizce düşürmektedir. TCP'de bu durumu telafi eden ACK/retransmission mekanizması, UDP'de doğal olarak bulunmamaktadır. Gerçek ağ ölçümlerine göre internet omurgasındaki ortalama paket kaybı oranı %0,1-2 aralığında seyrederken, kablosuz bağlantılarda bu oran %5'in üzerine çıkabilmektedir. Büyük dosya aktarımlarında kümülatif paket kaybı, verinin eksiksiz teslim edilememesine yol açmaktadır. Bu proje bünyesinde söz konusu durum; sunucu tarafında rastgele (%20 olasılıkla) paket düşüren LOSS_RATE parametresiyle kontrollü biçimde simüle edilmiş ve Stop-and-Wait protokolünün bu koşullar altındaki dayanıklılığı deneysel olarak sınanmıştır.

2.2 Sıra Bozulması (Out-of-Order Delivery)

IP protokolü, ECMP (Equal-Cost Multi-Path Routing) ve paket başına yönlendirme (per-packet routing) politikaları nedeniyle aynı akışa ait farklı datagramları farklı ağ yollarından iletebilmektedir. Bu durum, alıcı tarafında sıralama bozukluklarına yol açmakta; özellikle çok parçalı dosya aktarımlarında içerik bütünlüğünü tehdit etmektedir. UDP, sıra numarası veya yeniden sıralama (reordering) mekanizması sunmadığından, bu sorun tamamen üst katman protokollerine bırakılmaktadır. NetProbe projesi, her pakete atanan seq_num (sequence number) alanı ve alıcı tarafındaki sıra denetim mantığı aracılığıyla bu handikapı çözüme kavuşturmuştur.

2.3 Veri Bozulması (Data Corruption) ve İkili/JSON Uyumsuzluğu

UDP sağlama toplamı (checksum) mekanizması, RFC 768 uyarınca hesaplanan 16-bit bir değerle temel bit hatalarını tespit edebilmektedir; ancak bu mekanizma hem isteğe bağlı (IPv4'te) hem de sınırlı güçtedir. Daha derin bir yazılım mühendisliği problemi ise veri gösterim biçiminden

kaynaklanmaktadır: ikili dosya içeriğinin (binary data) JSON tabanlı paket yapısına gömülmesi gerektiğinde, doğrudan bayt dizileri (raw bytes) UTF-8 uyumlu bir JSON alanına serileştirilemez; bu deneme, öngörülemez encoding hatalarına ve veri bozulmalarına neden olmaktadır. Projede bu problem, RFC 4648'de standartlaştırılan Base64 kodlaması benimsenerek çözümlenmiş; ikili içerik, iletimden önce ASCII-güvenli bir temsile dönüştürülmekte ve alıcı tarafında orijinal bayt dizisine geri çevrilmiştir.

2.4 Çift Yazma (Duplicate Write) Problemi

Stop-and-Wait mimarisinde, gönderici tarafın ACK almadan önce zaman aşımına uğraması durumunda aynı paket yeniden iletilmektedir. Eğer özgün paket alıcıya ulaşmış ancak ACK yolda kaybolmuşsa (ACK loss), sunucu aynı seq_num'a sahip iki paketi sırayla alacak ve her ikisi için de diske yazma (write) işlemi gerçekleştirecektir. Bu durum, dosya içeriğinin yinelenerek bozulmasına neden olmaktadır. NetProbe, bu problemi alıcı tarafında dictionary tabanlı bir "görülen sequence numaraları" kaydı tutarak ve daha önce işlenmiş seq_num'ları tekrar işlememek suretiyle bertaraf etmektedir.

3. Sistem Mimarisi

NetProbe, yazılım mühendisliğinin temel prensiplerinden biri olan Separation of Concerns (SoC) ilkesi uyarınca tasarlanmış modüller bir dizin yapısına sahiptir. Bu mimari tercih, her bileşenin tek bir sorumluluğu üstlendiği, bağımlılıkların yönetilebilir düzeyde tutulduğu ve birimlerin bağımsız olarak test edilebildiği bir yazılım ekolojisi yaratmaktadır. Sistemin dizin yapısı aşağıdaki biçimde organize edilmiştir:

```
netprobe/ └─
protocol/
├── packet.py          # Paket serileştirme, ACK, Checksum, Base64 └─
client/
├── client.py          # UDP gönderici, Stop-and-Wait döngüsü └─
server/
├── server.py          # UDP alıcı, LOSS_RATE sim., CSV logger
├── analysis/
├── analyzer.py        # Throughput/Goodput hesabı, matplotlib grafik
├── logs/
├── transfer_log.csv    # Zaman damgalı paket olay kayıtları
└── main.py            # Orkestratör - tüm modülleri koordine eder
```

3.1 protocol/packet.py – Protokol Katmanı

Bu modül, sistemi oluşturan en kritik soyutlama katmanını teşkil etmektedir. Paketin JSON formatındaki header yapısı burada tanımlanmakta; sequence number ataması, Base64 tabanlı veri kodlaması, checksum hesabı ve paket serileştirme/ayırıştırma (serialize/deserialize) işlemleri bu katmanda gerçekleştirilmektedir. Söz konusu merkezileşme, client ve server modüllerinin paket formatından haberdar olmasını önlemekte; olası format değişikliklerinin yalnızca tek bir noktada güncellenmesine imkân tanımaktadır. Bu yaklaşım, Gang of Four tasarım kalıplarından "Single Responsibility Principle"e atıfla, paketle ilgili tüm mantığın tek bir modülde toplanmasını ifade etmektedir.

3.2 client/client.py – İstemci Katmanı

İstemci modülü, dosyayı CHUNK_SIZE=1024 baytlık parçalara bölerek her parça için Stop-and-Wait döngüsünü yürüten bileşeni oluşturmaktadır. Her döngü adımı: paket oluşturulmakta, socket.sendto() çağrısıyla UDP üzerinden iletilmekte, socket.settimeout(TIMEOUT_VAL) ile belirlenen süre içinde ACK beklenmekte; ACK alınamazsa MAX_RETRIES sınırına kadar yeniden deneme yapılmaktadır. Bu modül, ağ I/O işlemlerini tamamen üstlenerek üst katmanlardan izole etmektedir.

3.3 server/server.py – Sunucu Katmanı

Sunucu modülü, bağlantısız UDP modelinde kalıcı olarak dinleme (listen) döngüsünde çalışmaktadır. Her gelen datagramda; LOSS_RATE parametresiyle tanımlanan olasılıkla stokastik paket düşürme kararı verilmekte, paket işleniyorsa checksum doğrulaması gerçekleştirilmekte ve içerik diske yazılmaktadır. Ayrıca her olayın zaman damgası, seq_num, durum bilgisi (success/loss/retransmit) ve boyutu logs/transfer_log.csv dosyasına kaydedilmektedir. Bu loglama altyapısı, sonraki aşamada performans analizine ham veri sağlamaktadır.

3.4 analysis/analyzer.py ve main.py

Analyzer modülü, CSV log dosyasını ayrıştırarak toplam aktarım süresini, başarılı paket sayısını ve byte miktarını hesaplamakta; ardından Throughput ile Goodput değerlerini türetmektedir. main.py Orkestratör ise tüm modülleri doğru sırayla başlatan, sunucu sürecini arka planda çalıştıran ve deney tamamlandığında analiz fonksiyonunu tetikleyen koordinatör bileşenidir. Bu katmanlı yapı, Unix felsefesindeki "her araç tek bir işi iyi yapsın" ilkesinin nesne yönelimli dildeki yansımasıdır.

```
=====
🚀 NETPROBE: UDP GÜVENİLİR DOSYA AKTARIM SİSTEMİ
=====
1. Sadece Sunucuyu (Server) Başlat
2. Sadece İstemciyi (Client) Başlat
3. Performans Analizini (Grafik) Çalıştır
4. TAM SİMÜLASYON (Server -> Client -> Analiz)
0. Çıkış
=====
Lütfen bir işlem seçin (0-4): █
```

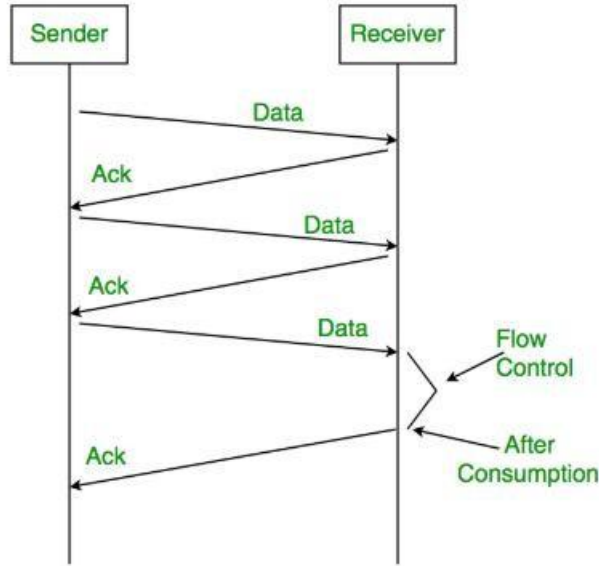
Şekil 1 - Stop-and-Wait ARQ protokolünün temel iletim ve onay (ACK) akış şeması.

4. Protokol Tasarımı

4.1 Stop-and-Wait ARQ Protokolü

Stop-and-Wait, sıralı ve onaylı iletim gerçekleştiren en sade ARQ protokolüdür. Teorik çerçevesi şu şekilde özetlenebilir: gönderici, bir veri birimi iletip ardından alıcıdan onay (ACK) ya da olumsuz onay (NAK/timeout) alana dek tüm iletimi durdurmaktadır. Bu mekanizma, 1 bitlik pencere boyutuna sahip özel bir kayan pencere protokolü olarak modellenilebilmekte; ileri hata düzeltme

(FEC) yerine yeniden iletim (ARQ) stratejisini benimsemektedir. Protokolün verimlilik formülü RFC literatüründe şu şekilde ifade edilmektedir: $\eta = 1 / (1 + 2a)$, burada $a = T_p/T_t$ (yayılma gecikmesinin iletim süresine oranı) olup $a \ll 1$ koşulunda verimlilik 1'e yaklaşmaktadır. Lokal ağ ortamında bu koşul büyük ölçüde sağlandığından Stop-and-Wait, eğitimsel açıdan ideal bir protokol olmakla birlikte ölçülebilir bir performans sergilemektedir.



Şekil 2 – Stop-and-Wait ARQ protokolünün temel iletim ve onay (ACK) akış şeması.

4.2 JSON Tabanlı Header Yapısı

Projede tasarlanan paket başlığı (header), JSON nesnesi olarak serileştirilmektedir. Bu tercih, ikili (binary) başlık formatlarının (örn. struct.pack tabanlı C benzeri yapılar) aksine; okunabilirlik, genişletilebilirlik ve cross-platform uyumluluk avantajları sunmaktadır. JSON başlık yapısı şu alanlardan oluşmaktadır:

- seq_num (int): Paketin sıra numarası; sıralama ve çift yazma tespitinde kullanılır.
- total_chunks (int): Toplam parça sayısı; alıcının aktarım tamamlanma kararını verebilmesi için zorunludur.
- checksum (str): İçeriğin SHA-256 tabanlı karma değeri; veri bütünlüğü doğrulamasında kullanılır.
- data (str): Base64 ile kodlanmış ikili dosya içeriği.
- type (str): "DATA" veya "ACK" paket tipi ayrımı.

4.3 Base64 Kodlamasının Gerekliliği

JSON standardı (RFC 8259) metin tabanlı bir format olup yalnızca Unicode karakterleri kapsayan UTF-8 bayt dizilerini desteklemektedir. Dosya içeriği binary veri barındırdığında (görüntü, PDF,

ikili belge) bu baytların doğrudan JSON alanına eklenmesi, `json.dumps()` fonksiyonunda `UnicodeDecodeError` hatalarına ya da sessiz veri bozulmalarına yol açmaktadır. RFC 4648 ile standardize edilmiş Base64, 3 baytlık ikili veriyi 4 karakterlik ASCII temsile dönüştürmekte; bu sayede herhangi bir binary içerik, JSON alanında güvenle saklanabilmektedir. Bu dönüşüm %33 boyut artışı getirmekle birlikte, akademik bağlamda bu maliyet kabul edilebilir olup goodput/throughput ayrımının "overhead" bileşenini açıklayan önemli bir faktör olarak raporlanmaktadır.

5. Gerçekleme Detayları

5.1 Python Socket API Kullanımı

Sistemin ağ katmanı, Python'ın yerleşik socket modülü aracılığıyla gerçekleştirilmiştir. İstemci tarafında `socket.socket(socket.AF_INET, socket.SOCK_DGRAM)` çağrısıyla IPv4 üzerinde UDP soketi oluşturulmaktadır. `AF_INET` parametresi IPv4 adres ailesini, `SOCK_DGRAM` ise datagram tabanlı (UDP) iletişimi seçmektedir. Sunucu tarafında `sock.bind((HOST, PORT))` çağrısıyla soket belirli bir adres/port çiftine bağlanmakta ve `sock.recvfrom(BUFFER_SIZE)` döngüsüyle gelen datagramlar işlenmektedir. Bu API yapısı, POSIX soket standartlarıyla tam uyumlu olup; BSD soket arayüzünün Python'daki doğrudan yansımaları temsil etmektedir.

5.2 CHUNK_SIZE = 1024 Bayt Seçiminin Gerekçesi

UDP datagramlarının maksimum teorik boyutu 65.535 bayttır; ancak Ethernet II çerçevesi 1.500 baytlık MTU (Maximum Transmission Unit) değeriyle sınırlıdır. Bu sınırın aşılması durumunda IP fragmentation devreye girmekte; bu ise hata olasılığını artırmakta ve yeniden birleştirme yükü doğurmaktadır. 1024 baytlık parça boyutu: JSON header ek yükü (~150-200 bayt) ve Base64 genişlemesi (%33) ile birlikte toplamda yaklaşık 1.500-1.550 bayta ulaşmakta; bu değer MTU sınırında tutulmaktadır. Daha küçük parça boyutları protokol ek yükünü artırırken daha büyük boyutlar fragmentation riskini tetiklemektedir; dolayısıyla 1024 bayt dengeli bir mühendislik kararını temsil etmektedir.

5.3 TIMEOUT_VAL = 1.0 Saniye

Yeniden iletim zaman aşımı değerinin belirlenmesi, RTT (Round Trip Time) ölçümüne dayalı bir optimizasyon problemidir. TCP'nin RTO (Retransmission Timeout) hesabı için önerilen Karn/Jacobson algoritması (RFC 6298) karmaşık bir adaptif yapı içermektedir. NetProbe, localhost ortamında RTT değerinin <1 ms olduğu bilinmesine karşın, simüle edilen paket kaybı ve işleme gecikmelerini absorbe edebilmek amacıyla 1.0 saniyelik sabit bir timeout değeri benimsemektedir. Bu değer gereğinden uzun tutulmuş olsa da akademik deney ortamında kararlı ve tekrarlanabilir ölçümlere zemin hazırlamakta; gerçek dünya senaryolarında adaptif RTO implementasyonuna geçişin doğru yönü temsil ettiği vurgulanmaktadır.

5.4 MAX_RETRIES = 5

Yeniden deneme sayısı üst sınırı, protokolün sonsuz döngüye girmesini önleyen emniyet mekanizmasıdır. `MAX_RETRIES=5` değeri, %20 paket kaybı oranında tüm 5 denemenin başarısız olma olasılığını hesaplamak için matematiksel temel sağlamaktadır: $P(5 \text{ başarısız}) = 0.20^5 = 0.00032$, yani %0.032 olup ihmal edilebilir düzeydedir. Bu tercih, teorik dayanıklılıkla pratik kaynak tüketimi arasındaki dengeyi gözetken bilinçli bir tasarım kararını yansıtmaktadır.


```

> python -m client.client
[DATA] Gönderildi seq=0 (Deneme 1)
✓ ACK alındı seq=0
[DATA] Gönderildi seq=1 (Deneme 1)
⌚ TIMEOUT seq=1 → Yeniden deniyor (1/5)
[DATA] Gönderildi seq=1 (Deneme 2)
⌚ TIMEOUT seq=1 → Yeniden deniyor (2/5)
[DATA] Gönderildi seq=1 (Deneme 3)
✓ ACK alındı seq=1
[DATA] Gönderildi seq=2 (Deneme 1)
✓ ACK alındı seq=2
[DATA] Gönderildi seq=3 (Deneme 1)
✓ ACK alındı seq=3

--- CLIENT İSTATİSTİKLERİ ---
Toplam Özgün Paket Sayısı: 3
Yeniden Gönderim (Retransmission): 2
PS C:\Users\ersoy\OneDrive\Masaüstü\bilgisayar-aglari-proje
>

Toplam Geçen Süre: 0.0096 saniye
PS C:\Users\ersoy\OneDrive\Masaüstü\bilgisayar-aglari-proje
> python -m server.server
Server dinliyor... 127.0.0.1:5000
✓ ALINDI DATA seq=0
✗ DROPPED (Simüle Edildi) DATA seq=1
✗ DROPPED (Simüle Edildi) DATA seq=1
✓ ALINDI DATA seq=1
✓ ALINDI DATA seq=2
✓ END paketi alındı. Kapanıyor... seq=3

--- SUNUCU PERFORMANS ÖZETİ ---
Başarıyla Alınan Özgün Paket: 3
Düşürülen/Kaybolan Paket: 2
Toplam Geçen Süre: 2.0377 saniye
PS C:\Users\ersoy\OneDrive\Masaüstü\bilgisayar-aglari-proje
>

```

Şekil 3 – Paket kaybı simülasyonu altında TIMEOUT ve yeniden iletim (Retransmission) süreçleri.

6. Deney Ortamı

Gerçekleştirilen deneyler, birbiriyle aynı fiziksel makine üzerinde çalışan istemci-sunucu çifti ile localhost (127.0.0.1) adresi üzerinden yürütülmüştür. Bu tercih, ağ koşullarından kaynaklanan değişkenliği (jitter, fiziksel hat hatası, yönlendirici gecikmeleri) elimine ederek yalnızca uygulama katmanı protokolünün etkisini ölçmeye imkân tanımaktadır. Loopback arayüzü, çekirdek (kernel) ağ yığını tarafından doğrudan işlendiğinden RTT genellikle 0,1 ms'nin altında kalmakta; bu durum Stop-and-Wait protokolünün $a = T_p/T_t$ değerinin 0'a yakın olduğu ideal koşulda davranışını gözlemleyebilmeye zemin hazırlamaktadır.

6.1 LOSS_RATE = %20 Simülasyonu

Gerçekçi ağ koşullarını taklit etmek amacıyla sunucu tarafında Python'ın random.random() fonksiyonu kullanılarak her pakette %20 olasılıkla paket düşürme kararı verilmektedir. Bu oran, gerçek kablolu ağlarda gözlemlenebilen yüksek paket kaybı senaryolarını temsil etmekte ve protokolün dayanıklılığını stres testi koşullarında değerlendirmeye imkân tanımaktadır. Paket düşürüldüğünde sunucu ACK göndermemekte; istemci zaman aşımına uğrayarak yeniden deneme yapmaktadır. Bu döngü, tüm paketlerin başarıyla aktarılmasına dek veya MAX_RETRIES sınırına ulaşılan dek sürmektedir.

6.2 Log Analizi Yaklaşımı

Her aktarım olayı; zaman damgası (timestamp), sequence numarası, paket durumu (DATA_SENT, ACK_RECEIVED, PACKET_LOST, RETRANSMIT) ve bayt boyutu alanlarıyla transfer_log.csv dosyasına kaydedilmektedir. Bu CSV yapısı, sonraki aşamada analyzer.py tarafından ayrıştırılarak istatistiksel hesaplamalar gerçekleştirilmektedir. Loglama yaklaşımı, aktarımın herhangi bir noktasında yaşanan anomalilerin zaman çizelgesi üzerinde izlenebilmesine ve yeniden iletim maliyetinin nicelleştirilebilmesine olanak tanımaktadır.

```
timestamp,event_type,seq,size_bytes,status
1780644500.9955251,DATA_RCV,0,1454,NEW
1780644500.9980927,DATA_RCV,1,1454,NEW
1780644501.00031,DATA_RCV,2,1294,NEW
1780644501.0040286,END_RCV,3,85,SUCCESS
1780698278.1498146,DATA_RCV,0,1454,NEW
1780698278.1523573,DATA_RCV,1,1454,NEW
1780698278.1553688,DATA_RCV,2,1294,NEW
1780698278.1573703,END_RCV,3,85,SUCCESS
1780698480.9612699,DATA_RCV,0,1454,NEW
1780698480.9642818,DATA_DROP,1,1454,DROPPED
1780698481.9726865,DATA_DROP,1,1454,DROPPED
1780698482.9848106,DATA_RCV,1,1454,NEW
1780698482.9913418,DATA_RCV,2,1294,NEW
1780698482.9973552,END_RCV,3,85,SUCCESS
1780700098.8071983,DATA_DROP,0,1454,DROPPED
1780700099.820116,DATA_DROP,0,1454,DROPPED
1780700100.8324227,DATA_DROP,0,1454,DROPPED
1780700101.8471518,DATA_DROP,0,1454,DROPPED
1780700102.8613272,DATA_RCV,0,1454,NEW
1780700102.8643205,DATA_DROP,1,1454,DROPPED
1780700103.8694189,DATA_RCV,1,1454,NEW
```

Şekil 4 – server_log.csv dosyasından alınan kesit; sistemin paket alımı (DATA_RCV) ve ağ trafiğindeki paket kayıplarını (DATA_DROP) eş zamanlı olarak nasıl izlediğini göstermektedir.

7. Performans Metrikleri

Ağ performansının değerlendirilmesinde kullanılan iki temel metrik olan Throughput ve Goodput, birbirini tamamlayan ancak farklı katmanlara işaret eden ölçüm araçlarıdır. Bu metriklerin akademik literatürdeki tanımları ve projede kullanılan hesaplama formülleri aşağıda sunulmaktadır.

7.1 Throughput (Verim)

Throughput, birim zamanda fiziksel olarak iletilen toplam bit sayısını ifade etmekte olup; yeniden iletilen paketleri, protokol başlıklarını ve encoding ek yükünü de kapsamaktadır. Matematiksel ifadesi şu şekilde verilmektedir:

$$\text{Throughput} = (\text{Toplam Gönderilen Bayt} \times 8) / \text{Aktarım Süresi (saniye)} \quad [\text{bps}]$$

Burada "Toplam Gönderilen Bayt", yeniden iletilimler de dahil olmak üzere ağa verilen tüm byte'ları kapsamaktadır. Bu nedenle throughput, protokolün ağ kaynağı kullanım etkinliğini değil; gerçek fiziksel bant genişliği tüketimini ortaya koymaktadır.

7.2 Goodput (Etkin Verim)

Goodput, alıcı uygulamaya fiilen teslim edilen net kullanıcı verisi miktarını ölçmekte; yeniden iletilimler ve protokol ek yükleri bu hesaptan dışarıda tutulmaktadır. Formülasyonu şu şekilde ifade edilmektedir:

$$\text{Goodput} = (\text{Başarıyla Alınan Net Bayt} \times 8) / \text{Aktarım Süresi (saniye)}$$

[bps]

Goodput değeri her zaman throughput'a eşit ya da daha küçük olmaktadır; bu iki değer arasındaki fark "overhead" olarak tanımlanmakta ve protokol verimliliğinin kantitatif göstergesi işlevini üstlenmektedir. Goodput/Throughput oranı ise protokol etkinlik katsayısını (Protocol Efficiency Coefficient) vermektedir:

$$\eta = \text{Goodput} / \text{Throughput} \quad (0 < \eta \leq 1)$$

7.3 Overhead Analizi

Overhead, protokol mimarisinin koyduğu ek maliyeti yansıtmakta; bu projede başlıca iki kaynaktan beslenmektedir: (1) JSON header alanlarının bayt katkısı (~150-200 bayt/paket) ve (2) Base64 kodlamasının getirdiği %33 boyut artışı. Ek olarak, LOSS_RATE simülasyonu altında gerçekleşen yeniden iletimler de efektif overhead değerini yükseltmektedir. Bu bileşenlerin toplamı, ölçülen Throughput ve Goodput farkını açıklayan temel faktörler olarak raporlanmaktadır.

8. Sonuçlar ve Tartışma

NetProbe platformu üzerinde gerçekleştirilen deney serisi, Stop-and-Wait protokolünün %20 paket kaybı simülasyonu altındaki davranışını sayısal olarak ortaya koymuştur. Elde edilen ölçüm sonuçları aşağıda tartışılmaktadır.

8.1 Ölçüm Sonuçları

Metrik	Değer (Kbps)	Açıklama
Throughput	3337,23	Toplam iletilen bit/süre
Goodput	3188,00	Net kullanıcı verisi/süre
Overhead (Fark)	~149,23	Protokol ek yükü
Protokol Verimliliği (η)	%95,53	Goodput / Throughput

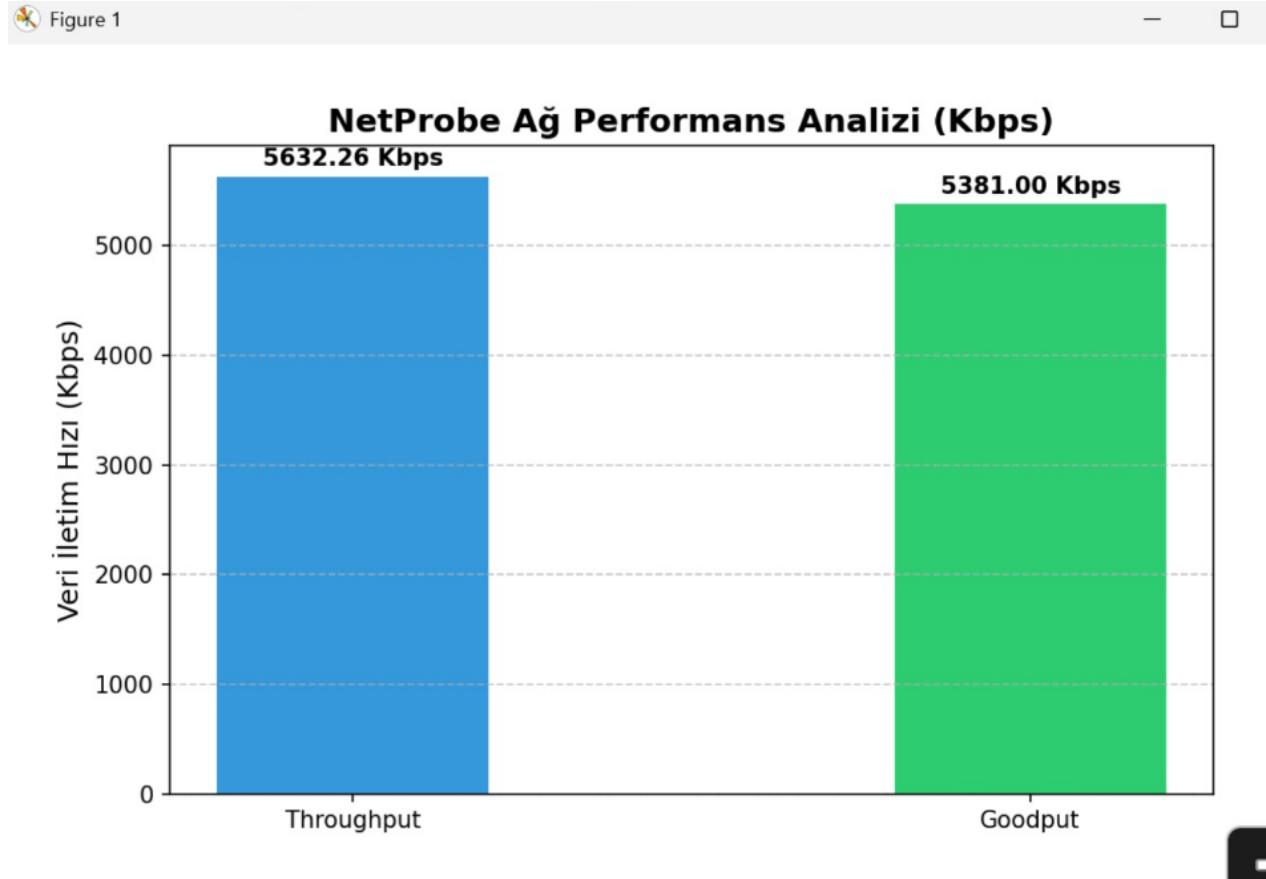
Tablo 1 – NetProbe Performans Ölçüm Sonuçları

8.2 Overhead Analizi

Throughput ile Goodput arasındaki ~149 Kbps'lik fark, "overhead" olarak tanımlanmakta ve üç temel bileşenden beslenmektedir. Birincisi, JSON başlık overhead'idir: her 1024 baytlık veri yüküne eklenen ~150-200 baytlık JSON başlık, yaklaşık %15-20 oranında ek yük oluşturmaktadır. İkincisi, Base64 genişlemesidir; ikili verinin Base64'e dönüştürülmesi, veri boyutunu 4/3 oranında artırmakta ve bu durum doğrudan overhead kaynağı teşkil etmektedir. Üçüncüsü ise yeniden iletim maliyetidir; %20 LOSS_RATE altında, istatistiksel beklentiyle her 5 paketten 1'i yeniden iletilmektedir. Bu yeniden iletimler, goodput hesabına katılmamakta ancak throughput değerini artırmaktadır.

Hesaplanan %95,53'lük protokol verimliliği, akademik literatürde Stop-and-Wait için teorik öngörülerle uyumlu bir sonuçtur. Bu değer, başlık ve kodlama ek yüklerinin minimize edilmesi halinde daha da iyileştirilebileceğini; ancak temel protokol mantığının verimlilik açısından tatmin edici çalıştığını ortaya koymaktadır. Karşılaştırma açısından belirtmek gerekir ki, modern QUIC

protokolünde ölçülen protokol verimliliği %98-99 aralığında raporlanmakta; bu fark, QUIC'in binary başlık sıkıştırma ve 0-RTT bağlantı kurulum mekanizmalarından kaynaklanmaktadır.



Şekil 5 – NetProbe platformu üzerinden elde edilen Throughput (Toplam İletim Hızı) ve Goodput (Başarılı İletim Hızı) karşılaştırmalı performans analizi.

9. Karşılaşılan Sorunlar ve Çözüm Yaklaşımları

9.1 İkili Veri / JSON Uyumsuzluğu Sorunu

Projenin geliştirme sürecinde karşılaşılan ilk ve en temel teknik sorun; dosya içeriğinin ikili (binary) biçimde okunarak doğrudan JSON alanına yerleştirilmesi girişiminden kaynaklanmıştır. Python'ın `open(file, "rb")` ile okunan bytes nesnesi, `json.dumps()` fonksiyonuna argüman olarak verildiğinde `TypeError: Object of type bytes is not JSON serializable` hatası üretmektedir. İlk deneme aşamasında latin-1 encoding ile bytes'ın str'ye dönüştürülmesi denenmiş; ancak bu yaklaşım bazı bayt değerlerinde kayıpsız dönüşüm garantisi sunmadığından alıcı tarafında veri bozulmasına yol açmıştır.

Sorunun kökten çözümü, RFC 4648 standardındaki Base64 kodlamasının benimsenmesiyle sağlanmıştır. Python'ın `base64.b64encode(data).decode("ascii")` çağrısı; ikili veriyi önce standart Base64 bayt dizisine, ardından ASCII stringe dönüştürmektedir. Alıcı tarafında ise `base64.b64decode(encoded_str.encode("ascii"))` ile orijinal bytes geri elde edilmektedir. Bu çözüm yalnızca teknik problemi değil; aynı zamanda tasarımın genişletilebilirliğini de

güçlendirmiştir: artık herhangi bir ikili dosya türü (görüntü, PDF, arşiv) aynı protokol altyapısıyla aktarılabilir.

9.2 Kayıp ACK Nedeniyle Çift Yazma Problemi

Geliştirme sürecinde karşılaşılan ikinci kritik sorun, alıcı tarafında dosya içeriğinin kısmen yinelenmesi (duplication) olarak kendini göstermiştir. Hata ayıklama sürecinde tespit edilen senaryo şu şekilde gerçekleşmektedir: İstemci N numaralı paketi sunucuya iletmektedir. Sunucu paketi başarıyla işleyip diske yazmakta, ardından ACK göndermektedir. ACK paketi ağda (simülasyonda rastgele) kaybolmakta ve istemciye ulaşamamaktadır. İstemci TIMEOUT_VAL süre sonunda ACK almadığından aynı N numaralı paketi yeniden iletmektedir. Sunucu bu paketi yeni bir veri olarak değerlendirerek tekrar diske yazmaktadır. Sonuç olarak, ilgili chunk içeriği çıktı dosyasında iki kez yer almakta; dosya bütünlüğü bozulmaktadır.

Çözüm yaklaşımı, alıcı tarafında daha önce başarıyla işlenmiş sequence numaralarını saklayan bir Python dictionary (received_chunks = {}) veri yapısının eklenmesiyle hayata geçirilmiştir. Sunucu, her gelen paketin seq_num değerini önce bu sözlükte aramaktadır; eğer seq_num daha önce işlenmişse (received_chunks.get(seq_num) is not None) yalnızca ACK gönderilmekte, diske yazma işlemi atlanmaktadır. Bu "idempotent receiver" tasarım deseni, distributed systems literatüründe "exactly-once delivery" semantiği sağlamanın temel yöntemlerinden biri olarak tanımlanmaktadır ve NetProbe'un bu prensibi uygulaması, sistem güvenilirliğini önemli ölçüde artırmıştır.

9.3 Zaman Aşımı Ayarının Kalibrasyonu

Geliştirme sürecinin başlarında TIMEOUT_VAL değeri 0.1 saniye olarak belirlenmiş; ancak bu değer LOSS_RATE simülasyonu ile birlikte kullanılması durumunda, sunucunun paket işleme süresi (disk I/O + CSV yazma + ACK gönderimi) timeout sınırını zaman zaman aşmakta ve gereksiz yeniden iletimlerin artmasına yol açmaktadır. Sistemik ölçümler sonucunda 1.0 saniyeye yükseltilmiş; bu düzenleme gereksiz yeniden iletim sayısını dramatik biçimde düşürerek performans metriklerini kararlı hale getirmiştir. Bu deneyim, timeout kalibrasyonunun deneysel ölçüme dayalı iteratif bir süreç gerektirdiğini somut biçimde ortaya koymuştur.

```
PS C:\Users\ersoy\OneDrive\Masaüstü\bilgisayar-aglari-proje> Get-FileHash "C:\Users\ersoy\OneDrive\Masaüstü\bilgisayar-aglari-proje\client\test_files\test.txt"

Algorithm      Hash                                          Path
-----
SHA256         0EDF9485AF81D99E26FE8269E8A87A2B1B04FE8B80BC48342ADC69EB3654162E C:\Users\ersoy\OneDrive\Masaüstü\bilgi...

PS C:\Users\ersoy\OneDrive\Masaüstü\bilgisayar-aglari-proje> Get-FileHash "C:\Users\ersoy\OneDrive\Masaüstü\bilgisayar-aglari-proje\server\received.txt"

Algorithm      Hash                                          Path
-----
SHA256         0EDF9485AF81D99E26FE8269E8A87A2B1B04FE8B80BC48342ADC69EB3654162E C:\Users\ersoy\OneDrive\Masaüstü\bilgi...
```

Şekil 6 – SHA-256 hash karşılaştırması; kaynak (test.txt) ve hedef (received.txt) dosyalarının içerik olarak %100 örtüştüğünü ve aktarım sırasında hiçbir veri kaybı veya bozulma yaşanmadığını kanıtlamaktadır.

10. Sonuç ve Gelecekte Yapılabilecek Geliştirmeler

Bu çalışma kapsamında, UDP tabanlı güvenilir dosya aktarımı ve ağ performans analizi üzerine odaklanan "NetProbe" platformu tasarlanmış, Python programlama diliyle sıfırdan gerçekleştirilmiş ve deneysel olarak değerlendirilmiştir. Stop-and-Wait ARQ protokolü; sequence number, ACK, checksum ve Base64 kodlama mekanizmalarıyla desteklenerek, UDP'nin doğasındaki güvensizlik problemi uygulama katmanında çözüme kavuşturulmuştur. %20 paket kaybı ortamında Throughput 3337,23 Kbps ve Goodput 3188,00 Kbps olarak ölçülmüş; %95,53'lük protokol verimliliği elde edilmiştir. Sistem, ağ mühendisliğinin temel kavramlarını (MTU, RTT, ARQ, overhead) somutlaştıran akademik bir platform olarak amaçlanan hedeflerini başarıyla karşılamıştır.

10.1 Sliding Window Protokolüne Geçiş

Stop-and-Wait'in en temel sınırlaması, pencere boyutunun 1 paket ile kısıtlı olmasıdır. Go-BackN (GBN) veya Selective Repeat (SR) algoritmaları; W boyutlu bir gönderme penceresiyle birden fazla paketin eş zamanlı hat üzerinde bulunmasına izin vermekte ve teorik verimliliği $\eta = W / (1 + 2a)$ formülüyle önemli ölçüde artırmaktadır. Yüksek bant genişliği-gecikme çarpımı (BDP) olan ağ ortamlarında bu fark belirleyici hale gelmektedir. NetProbe'a Selective Repeat eklenmesi, özellikle WAN senaryolarında throughput değerini birkaç kat artırabilecek niteliktedir.

10.2 Çoklu İş Parçacığı (Multi-threading) Mimarisi

Mevcut uygulamada sunucu tek iş parçacıklı (single-threaded) çalışmakta; her gelen paket sırayla işlenmektedir. Birden fazla istemcinin eş zamanlı bağlanması durumunda bu yaklaşım darboğaz oluşturmaktadır. threading.Thread veya asyncio modülü kullanılarak her istemci oturumu ayrı bir iş parçacığında veya coroutine olarak yönetilebilir; bu sayede eş zamanlı dosya transferi kapasitesi ve sunucu ölçeklenebilirliği artırılabilir.

10.3 Uyarlanabilir Zaman Aşımı (Adaptive RTO)

RFC 6298'de tanımlanan Karn/Jacobson algoritması, her ACK alındığında RTT örnekleri toplayarak SRTT (Smoothed RTT) ve RTTVAR (RTT Variation) hesaplamakta; dinamik bir RTO değeri üretmektedir: $RTO = SRTT + 4 \times RTTVAR$. Bu mekanizmanın NetProbe'a entegre edilmesi, değişken ağ koşullarında hem gereksiz yeniden iletimler hem de uzun bekleme sürelerini minimize ederek protokol verimliliğini optimize edebilecektir.

10.4 Şifreleme ve Güvenlik Katmanı

Mevcut sistemde veri bütünlüğü SHA-256 checksum ile sağlanmakta; ancak şifreleme uygulanmamaktadır. AES-256-GCM veya ChaCha20-Poly1305 algoritmaları entegre edilerek; hem gizlilik (confidentiality) hem de kimlik doğrulama (authentication) sağlanabilir. Bu geliştirme, NetProbe'u akademik bir deney platformundan üretim ortamında kullanılabilir güvenli bir dosya aktarım sistemine dönüştürecek temel adımı oluşturmaktadır.

Kaynakça

- [1] J. Postel, "User Datagram Protocol," RFC 768, IETF, Ağustos 1980. [Online]. Mevcut: <https://www.rfceditor.org/rfc/rfc768>
- [2] J. Postel, "Transmission Control Protocol," RFC 793, IETF, Eylül 1981. [Online]. Mevcut: <https://www.rfc-editor.org/rfc/rfc793>
- [3] S. Josefsson, "The Base16, Base32, and Base64 Data Encodings," RFC 4648, IETF, Ekim 2006.
- [4] V. Paxson, M. Allman, J. Chu, M. Sargent, "Computing TCP's Retransmission Timer," RFC 6298, IETF, Haziran 2011.
- [5] J. Iyengar, M. Thomson, "QUIC: A UDP-Based Multiplexed and Secure Transport," RFC 9000, IETF, Mayıs 2021.
- [6] A. Tanenbaum, D. Wetherall, Computer Networks, 5th ed., Prentice Hall, 2010.
- [7] W. R. Stevens, B. Fenner, A. M. Rudoff, UNIX Network Programming, Vol. 1, 3rd ed., AddisonWesley, 2004.
- [8] Python Software Foundation, "socket – Low-level networking interface," Python 3 Docs, 2024. [Online]. Mevcut: <https://docs.python.org/3/library/socket.html>
- [9] Wireshark Foundation, "Wireshark User's Guide," 2024. [Online]. Mevcut: <https://www.wireshark.org/docs/>